SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	O.Siddirishwanth	**Reg. No.:**	192472148
Section / Batch:	B. Tech AI&ML (2024-2028)	**Date:**	12/08/2026

Code of Conduct

I O.Siddirishwanth(192472148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1:

Design and implement a Python-based N-gram language model that predicts the next word in a sentence using a given English text corpus. The system should preprocess the corpus, tokenize the sentences, count unigram, bigram, and trigram occurrences, and calculate their probabilities using an unsmoothed N-gram model. Given an incomplete sentence such as “The student is”, the system should identify and display the most probable next words.

The program should provide options to select the value of N (1, 2, or 3), display the corresponding wordfrequency counts and probabilities, and generate the top-5 next-word predictions. The implementation should also demonstrate how unseen N-grams result in zero probability. Finally, evaluate the prediction performance using suitable test sentences and discuss the limitations of an unsmoothed N-gram model. **Python Program**

```
from collections import Counter import
re

corpus = """
the student is studying natural language processing
the student is reading a book the student likes
machine learning
the teacher is teaching natural language processing the
teacher likes machine learning
students are learning natural language processing
""" tokens = re.findall(r'\b\w+\b',

corpus.lower())

def get_ngrams(tokens, n):     return
[tuple(tokens[i:i+n])             for i in
range(len(tokens)-n+1)]

unigrams = get_ngrams(tokens, 1)
bigrams = get_ngrams(tokens, 2) trigrams
= get_ngrams(tokens, 3)

uni_count = Counter(unigrams)
bi_count = Counter(bigrams) tri_count
= Counter(trigrams)

print("===== UNIGRAM COUNTS =====")
for gram, count in uni_count.items():
print(gram, ":", count)

print("\n===== BIGRAM COUNTS =====")
for gram, count in bi_count.items():
print(gram, ":", count)

print("\n===== TRIGRAM COUNTS
=====") for gram, count in tri_count.items():
print(gram, ":", count)

def bigram_probability(w1, w2):
count_bigram = bi_count[(w1, w2)]
count_word = uni_count[(w1,)]

   if count_word == 0:
return 0

   return count_bigram / count_word

def trigram_probability(w1, w2, w3):
count_trigram = tri_count[(w1, w2, w3)]
count_bigram = bi_count[(w1, w2)]
```

```
    if count_bigram == 0:
return 0

    return count_trigram / count_bigram

def predict_next(words, n):
words = words.lower().split()

    if n == 1:
        candidates = []         for (word,), count in
uni_count.items():              probability = count /
len(tokens)             candidates.append((word,
probability))

    elif n == 2:
previous = words[-1]
candidates = []

        for (w1, w2), count in bi_count.items():
if w1 == previous:                  probability =
count / uni_count[(w1,)]
candidates.append((w2, probability))

    elif n == 3:         previous =
tuple(words[-2:])         candidates
= []

        for (w1, w2, w3), count in tri_count.items():
if (w1, w2) == previous:                  probability =
count / bi_count[(w1, w2)]
candidates.append((w3, probability))

    else:
return []

    candidates.sort(key=lambda x: x[1], reverse=True)
return candidates[:5] query = "the student is"

print("\n===== TOP-5 NEXT WORD PREDICTIONS =====")

for n in [1, 2, 3]:     print(f"\nN =
{n}")     predictions =
predict_next(query, n)

    if predictions:         for word, probability
in predictions:             print(f"{word:15}
{probability:.4f}")     else:
        print("No prediction available.")

print("\n===== UNSEEN BIGRAM =====")

probability = bigram_probability("student", "football") print("P(football

| student) =", probability)
```

```
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 % /usr/local/bin
===== UNIGRAM COUNTS =====
('the',) : 5
('student',) : 3
('is',) : 3
('studying',) : 1
('natural',) : 3
('language',) : 3
('processing',) : 3
('reading',) : 1
('a',) : 1
('book',) : 1
('likes',) : 2
('machine',) : 2
('learning',) : 3
('teacher',) : 2
('teaching',) : 1
('students',) : 1
('are',) : 1

===== BIGRAM COUNTS =====
('the', 'student') : 3
('student', 'is') : 2
('is', 'studying') : 1
('studying', 'natural') : 1
('natural', 'language') : 3
('language', 'processing') : 3
('processing', 'the') : 2
('is', 'reading') : 1
('reading', 'a') : 1
('a', 'book') : 1
('book', 'the') : 1
('student', 'likes') : 1
('likes', 'machine') : 2
('machine', 'learning') : 2
('learning', 'the') : 1
('the', 'teacher') : 2
('teacher', 'is') : 1
('is', 'teaching') : 1
('teaching', 'natural') : 1
('teacher', 'likes') : 1
('learning', 'students') : 1
('students', 'are') : 1
('are', 'learning') : 1
('learning', 'natural') : 1
```

```
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 % /usr/local/bi

===== TRIGRAM COUNTS =====
('the', 'student', 'is') : 2
('student', 'is', 'studying') : 1
('is', 'studying', 'natural') : 1
('studying', 'natural', 'language') : 1
('natural', 'language', 'processing') : 3
('language', 'processing', 'the') : 2
('processing', 'the', 'student') : 1
('student', 'is', 'reading') : 1
('is', 'reading', 'a') : 1
('reading', 'a', 'book') : 1
('a', 'book', 'the') : 1
('book', 'the', 'student') : 1
('the', 'student', 'likes') : 1
('student', 'likes', 'machine') : 1
('likes', 'machine', 'learning') : 2
('machine', 'learning', 'the') : 1
('learning', 'the', 'teacher') : 1
('the', 'teacher', 'is') : 1
('teacher', 'is', 'teaching') : 1
('is', 'teaching', 'natural') : 1
('teaching', 'natural', 'language') : 1
('processing', 'the', 'teacher') : 1
('the', 'teacher', 'likes') : 1
('teacher', 'likes', 'machine') : 1
('machine', 'learning', 'students') : 1
('learning', 'students', 'are') : 1
('students', 'are', 'learning') : 1
('are', 'learning', 'natural') : 1
('learning', 'natural', 'language') : 1

===== TOP-5 NEXT WORD PREDICTIONS =====

N = 1
the             0.1389
student         0.0833
is              0.0833
natural         0.0833
language        0.0833

N = 2
studying        0.3333
reading         0.3333
teaching        0.3333

N = 3
studying        0.5000
reading         0.5000
```

```
===== UNSEEN BIGRAM =====
P(football | student) = 0.0
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 %
```

Task 2:

Design and implement a Python-based language prediction system that improves an unsmoothed N-gram model using smoothing and backoff techniques. The system should initially construct a bigram/trigram model from an English corpus and identify situations where an N-gram has zero probability because it does not occur in the training data.

Design the system to apply a backoff strategy, where the model uses trigram probability when available, otherwise backs off to the bigram probability and finally to the unigram probability. Extend the implementation using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using suitable interpolation weights. The program should accept a user-entered sentence/query and return the most probable next word. Compare the prediction results of the unsmoothed, backoff, and deleted-interpolation models. **Python Program**

```
from collections import Counter import
re
```

```
corpus = """ the student
is studying the student is
reading the student likes
books the teacher is
teaching the teacher
likes books the student
likes learning
""" tokens = re.findall(r'\b\w+\b',

corpus.lower()) unigram = Counter(tokens)

bigram = Counter(
    (tokens[i], tokens[i+1])
    for i in range(len(tokens)-1)
)

trigram = Counter(
    (tokens[i], tokens[i+1], tokens[i+2])
for i in range(len(tokens)-2)
)

def unigram_prob(word):
    return unigram[word] / len(tokens)

def bigram_prob(w1, w2):
    denominator = unigram[w1]

    if denominator == 0:
return 0

    return bigram[(w1, w2)] / denominator

def trigram_prob(w1, w2, w3):
denominator = bigram[(w1, w2)]

    if denominator == 0:
return 0

    return trigram[(w1, w2, w3)] / denominator

def backoff(w1, w2, w3):     p3 =
trigram_prob(w1, w2, w3)

    if p3 > 0:         return
p3, "Trigram"

    p2 = bigram_prob(w2, w3)

    if p2 > 0:         return
p2, "Bigram"

    p1 = unigram_prob(w3)

return p1, "Unigram"

def interpolation(w1, w2, w3):
```

```
    lambda3 = 0.5
lambda2 = 0.3      lambda1
= 0.2

    p3 = trigram_prob(w1, w2, w3)
p2 = bigram_prob(w2, w3)      p1
= unigram_prob(w3)

    probability = (
lambda3 * p3 +
lambda2 * p2 +
        lambda1 * p1
    )

    return probability context

= ("the", "student")

vocabulary = set(tokens)

print("===== PREDICTIONS =====")

results = []

for word in vocabulary:      p_tri =

trigram_prob(context[0], context[1], word)

    p_back, model = backoff(
context[0],         context[1],
        word
    )

    p_interp = interpolation(
context[0],         context[1],
        word
    )

    results.append(
        (word, p_tri, p_back, model, p_interp)
    )

results.sort(key=lambda x: x[4], reverse=True)

for word, p_tri, p_back, model, p_interp in results[:5]:

    print("\nWord:", word)      print("Unsmoothed
Trigram:", round(p_tri, 4))      print("Backoff:",
round(p_back, 4),            "using", model)
print("Interpolation:", round(p_interp, 4))
```

```
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 % /usr/local/bin/python3 /Users/bhanu/COD
===== PREDICTIONS =====

Word: likes
Unsmoothed Trigram: 0.5
Backoff: 0.5 using Trigram
Interpolation: 0.425

Word: is
Unsmoothed Trigram: 0.5
Backoff: 0.5 using Trigram
Interpolation: 0.425

Word: the
Unsmoothed Trigram: 0.0
Backoff: 0.25 using Unigram
Interpolation: 0.05

Word: student
Unsmoothed Trigram: 0.0
Backoff: 0.1667 using Unigram
Interpolation: 0.0333

Word: books
Unsmoothed Trigram: 0.0
Backoff: 0.0833 using Unigram
Interpolation: 0.0167
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 %
```

Task 3:

Design and implement a Python program to evaluate an English N-gram language model using the concept of entropy. Given a training corpus and a separate test corpus, construct unigram, bigram, and trigram models and calculate the probability of words occurring in the test sentences. Based on these probabilities, compute the entropy of the language model and compare the uncertainty associated with different N-gram models.

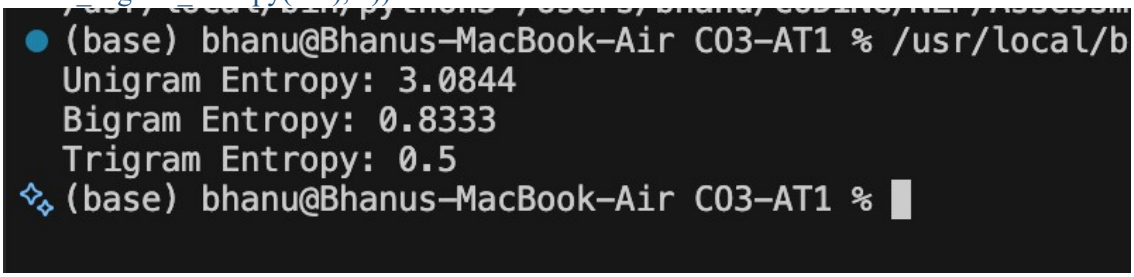
The application should simulate a text-prediction scenario in which the system receives a sequence of words and estimates how predictable the next word is. The program should identify sentences or word sequences having high and low entropy and provide an interpretation of the results. The system should also compare the effect of smoothing on entropy and explain why a model with better probability estimation can provide more reliable predictions.

Python Program

```
from collections import Counter
import math import re

train_text = """
the student is reading a book the
student is learning python the
teacher is reading a book the
teacher is teaching python
"""

test_text = """ the
student is learning the
teacher is reading
"""
```

```
train = re.findall(r'\b\w+\b', train_text.lower())
test = re.findall(r'\b\w+\b', test_text.lower())
unigram = Counter(train)

bigram = Counter(
    (train[i], train[i+1])
    for i in range(len(train)-1)
)

trigram = Counter(
    (train[i], train[i+1], train[i+2])
    for i in range(len(train)-2)
)

def calculate_unigram_entropy(test):
    entropy = 0
count = 0     for
word in test:

        probability = unigram[word] / len(train)

        if probability > 0:
            entropy -= math.log2(probability)
count += 1     return entropy / count


def calculate_bigram_entropy(test):
    entropy = 0     count = 0
for i in range(1, len(test)):

        previous = test[i-1]
current = test[i]

        denominator = unigram[previous]
numerator = bigram[(previous, current)]
if denominator > 0 and numerator > 0:
probability = numerator / denominator

            entropy -= math.log2(probability)
count += 1     return entropy / count

def calculate_trigram_entropy(test):
entropy = 0     count = 0     for i in
range(2, len(test)):

        w1 = test[i-2]
w2 = test[i-1]         w3
= test[i]

        denominator = bigram[(w1, w2)]
numerator = trigram[(w1, w2, w3)]         if
```

```
denominator > 0 and numerator > 0:

probability = numerator / denominator

        entropy -= math.log2(probability)
count += 1      return entropy / count


print("Unigram Entropy:",
round(calculate_unigram_entropy(test), 4))

print("Bigram Entropy:",
round(calculate_bigram_entropy(test), 4))

print("Trigram Entropy:",
round(calculate_trigram_entropy(test), 4))
```



Task 4

Design and implement a Python-based Part-of-Speech (POS) tagging system that assigns grammatical categories such as noun, verb, adjective, adverb, pronoun, preposition, and conjunction to words in an English sentence. The system should first implement a Rule-Based POS Tagger using lexical dictionaries and grammatical rules. Next, design a Stochastic POS Tagger using word/tag probabilities and transition probabilities derived from a training corpus.

Extend the system by implementing the basic idea of Transformation-Based Tagging, where initially assigned tags are corrected using transformation rules. For example, a word initially tagged as a noun may be transformed into a verb when it occurs after a pronoun or auxiliary verb. The final system should compare the output of the three approaches for the same input sentences and identify which approach provides better tagging results. The program should also demonstrate the use of appropriate English tagsets, such as the Penn Treebank tagset. **Python Program**

```
import re from collections import Counter,

defaultdict sentence = "The student is

reading a book" words =

sentence.lower().split() def

rule_based_tagger(words):

   tags = []

   for word in words:

      if word in ["the", "a", "an"]:
tag = "DT"

      elif word in ["is", "am", "are", "was", "were"]:
tag = "VBZ"

      elif word.endswith("ing"):
         tag = "VBG"
```

```
        elif word.endswith("ly"):
tag = "RB"

        elif word in ["i", "you", "he", "she", "we", "they"]:
tag = "PRP"

        else:             tag = "NN"

tags.append((word, tag))

return tags

training_data = [
    ("the", "DT"),
    ("student", "NN"),
("teacher", "NN"),
    ("is", "VBZ"),
    ("reading", "VBG"),
    ("writing", "VBG"),
    ("a", "DT"),
    ("book", "NN"),
    ("quickly", "RB")

]

word_tags = defaultdict(Counter)

for word, tag in training_data:

word_tags[word][tag] += 1 def

stochastic_tagger(words):

    result = []

    for word in words:

        if word in word_tags:
            tag = word_tags[word].most_common(1)[0][0]
else:             tag = "NN"         result.append((word,
tag))     return result

def transformation_tagger(words):

    result = [(word, "NN") for word in words]

    for i, (word, tag) in enumerate(result):
if word in ["the", "a", "an"]:
result[i] = (word, "DT")

        elif word in ["is", "am", "are", "was", "were"]:
result[i] = (word, "VBZ")
```

```
        elif word.endswith("ing"):              result[i]
= (word, "VBG")

        elif word.endswith("ly"):               result[i]
= (word, "RB")      return result

print("Sentence:") print(sentence)

print("\nRule-Based POS Tagging:") for word,
tag in rule_based_tagger(words):
print(word, "->", tag)

print("\nStochastic POS Tagging:") for word,
tag in stochastic_tagger(words):     print(word,
"->", tag)

print("\nTransformation-Based Tagging:") for
word, tag in transformation_tagger(words):
    print(word, "->", tag)
```

```
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 % /usr/local/bin/pythor
Sentence:
The student is reading a book

Rule-Based POS Tagging:
the -> DT
student -> NN
is -> VBZ
reading -> VBG
a -> DT
book -> NN

Stochastic POS Tagging:
the -> DT
student -> NN
is -> VBZ
reading -> VBG
a -> DT
book -> NN

Transformation-Based Tagging:
the -> DT
student -> NN
is -> VBZ
reading -> VBG
a -> DT
book -> NN
(base) bhanu@Bhanus-MacBook-Air CO3-AT1 %
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							

3						
4						

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____